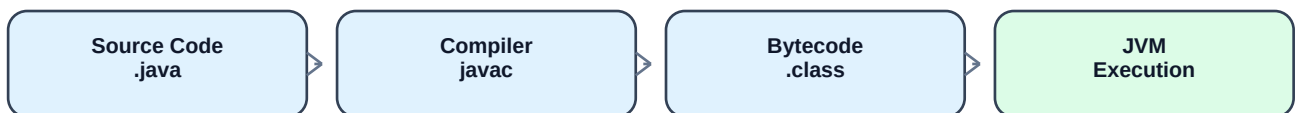


CORE JAVA

Basic Theory & Beginner's Learning Guide



What this guide covers

Java fundamentals, program structure, data types, variables, operators, input/output, control flow, methods, arrays, strings, OOP foundations, exceptions, packages, JDK/JRE/JVM, installation on Windows/macOS/Linux, and how a Java program actually runs.

Designed for someone starting Core Java from zero — with simple explanations, diagrams, tables, and executable examples.

Table of Contents

#	Topic
1	Java at a Glance
2	JDK, JRE and JVM
3	How Java Code Runs
4	Installing Java — Windows, macOS and Linux
5	Your First Java Program
6	Java Program Structure
7	Variables and Data Types
8	Type Casting
9	Operators
10	Input and Output
11	Control Flow
12	Methods
13	Arrays and Strings
14	Object-Oriented Programming Foundations
15	Packages and Access Modifiers
16	Exception Handling
17	Common Beginner Mistakes
18	Practice Roadmap

1. Java at a Glance

Java is a general-purpose, class-based programming language designed around the idea that compiled Java code can run on different operating systems through the Java Virtual Machine (JVM).

Why beginners learn Java

- Strong typing makes many programming mistakes easier to catch early.
- Object-oriented design is widely used in enterprise applications.
- A mature ecosystem exists for web, backend, data, cloud, testing, and tooling.
- The same Java bytecode can run on different operating systems when a compatible JVM is available.

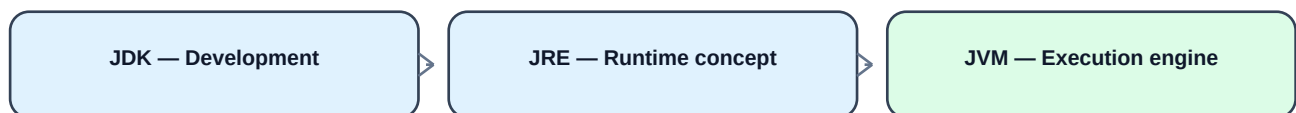
Java editions — simplified

Term	Meaning
Java SE	Standard Edition — core language, collections, I/O, concurrency, JDBC APIs, etc.
Jakarta EE	Enterprise specifications and technologies used for large server-side applications.
Java ME	A Java platform family historically targeted at constrained devices.

Important: Java is not the same thing as JavaScript. They are separate languages with different runtimes and ecosystems.

2. JDK, JRE and JVM

Think of these as layers rather than three unrelated products.



JDK: tools needed to develop Java applications, including the compiler and launcher.

JRE: a runtime concept consisting of the JVM plus runtime libraries/components.

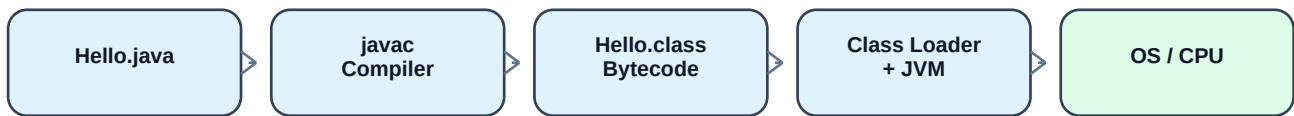
JVM: the engine that loads and executes Java bytecode.

Modern Java note: Developers normally install a JDK. You generally do not need to install a separate JRE package.

Component	Main job	Beginner example
JDK	Develop + compile + run	javac, java
JRE	Runtime environment concept	Libraries + JVM
JVM	Execute bytecode	HotSpot JVM

3. How Java Code Runs

Java source is compiled into bytecode. The JVM then loads that bytecode and executes it on the host operating system/CPU.



Step-by-step

- You write source code in a .java file.
- javac checks the source and produces .class bytecode.
- The JVM loads required classes.
- The JVM verifies/prepares bytecode and manages runtime execution.
- The JVM interprets code and can JIT-compile frequently executed code into native instructions.
- The OS and CPU execute the resulting instructions.

The JVM is the portability layer: Windows, macOS and Linux have JVM implementations for their platforms.

Example command flow

```
javac Hello.java
java Hello
```

The run command normally uses the class name (Hello), not Hello.class.

Stage	Command / Component	Output / Action
Write	Text editor / IDE	Hello.java
Compile	javac Hello.java	Hello.class
Run	java Hello	JVM starts application
Optimize	JIT inside JVM	Hot code may be compiled to native instructions

4. Installing Java — Windows, macOS and Linux

For learning Core Java, install a current LTS JDK distribution. Common choices include OpenJDK builds and Oracle JDK.

A. Windows

- Download a Windows JDK installer from a reputable JDK distribution provider.
- Install the JDK.
- Open Command Prompt or PowerShell and verify:

```
java -version
javac -version
```

If both commands return a version, your terminal can find Java and the compiler.

PATH and JAVA_HOME

- **PATH** lets the shell find commands such as java and javac.

- **JAVA_HOME** is commonly used by build tools to point to the JDK.

B. macOS

- Install a JDK package appropriate for Apple Silicon or Intel.
- Open Terminal and verify:

```
java -version
javac -version
/usr/libexec/java_home -V
```

Homebrew is another common installation route on macOS if you already use it.

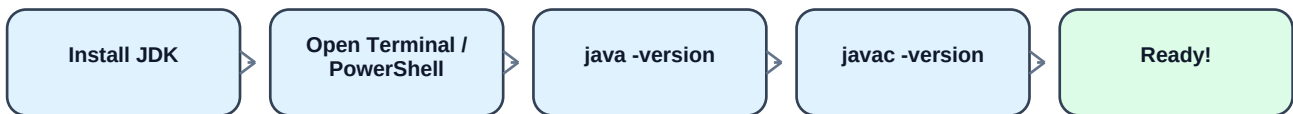
C. Linux

- Use your distribution's package manager when possible.
- For Debian/Ubuntu-based systems, for example:

```
sudo apt update
sudo apt install openjdk-21-jdk
java -version
javac -version
```

- For Fedora/RHEL-family systems, package names differ; use the distribution's package manager.

Installation verification



Which version should I learn? Java 21 LTS is a strong baseline for learning modern Core Java. If your course/employer specifies another version, use that environment.

5. Your First Java Program

Create a file named **HelloWorld.java**:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Compile and run:

```
javac HelloWorld.java
java HelloWorld
```

Expected output:

```
Hello, Java!
```

For a public top-level class, the source file name conventionally matches the class name.

6. Java Program Structure

Part	Example	Purpose
Class	class HelloWorld	Defines a type / blueprint.
main method	public static void main(String[] args)	Common entry point for a standalone application.
Statement	System.out.println(...);	An instruction executed by the program.
String literal	"Hello, Java!"	Text value enclosed in double quotes.
Semicolon	;	Terminates many Java statements.

Comments

```
// Single-line comment

/*
   Multi-line comment
*/

/**
 * Documentation comment (Javadoc)
 */
```

7. Variables and Data Types

A variable is a named storage location associated with a value. Java is statically typed: a variable has a declared type.

Primitive data types

Type	Typical size	Example	Use
byte	8-bit	byte age = 26;	Small integers
short	16-bit	short x = 1000;	Small/medium integers
int	32-bit	int salary = 50000;	Default integer choice
long	64-bit	long views = 100000L;	Large integers
float	32-bit	float pi = 3.14f;	Floating point
double	64-bit	double price = 99.99;	Default decimal choice
char	16-bit	char grade = 'A';	Single UTF-16 code unit
boolean	logical	boolean active = true;	true / false

Reference types

- Examples include String, arrays, classes, interfaces, enums, and objects.
- A reference variable refers to an object rather than containing the whole object directly.

```
int age = 26;
double salary = 50000.50;
char grade = 'A';
boolean active = true;
String name = "Nisarg";
```

Declaration vs initialization

```
int age;           // declaration
```

```
age = 26;           // assignment

int marks = 95; // declaration + initialization
```

Constants are commonly declared with final:

```
final double PI = 3.141592653589793;
```

8. Type Casting

Casting converts a value from one type to another. Widening conversions generally happen automatically; narrowing conversions normally require an explicit cast.



Widening:

```
int x = 10;
double y = x;
```

Narrowing:

```
double price = 99.99;
int whole = (int) price; // 99
```

Casting a floating-point value to an integer discards the fractional part; it does not round to the nearest integer.

9. Operators

Category	Operators	Example
Arithmetic	+ - * / %	a + b
Assignment	= += -= *= /= %=	x += 5
Comparison	== != > < >= <=	age >= 18
Logical	&& !	adult && active
Increment/decrement	++ --	count++
Ternary	? :	age >= 18 ? "Adult" : "Minor"
Bitwise	& ^ ~ << >> >>>	a & b

Parentheses make intent clearer and reduce precedence-related mistakes.

```
int result = (10 + 5) * 2;
boolean eligible = age >= 18 && active;
```

10. Input and Output

Output with System.out

```
System.out.print("Hello");
```

```
System.out.println("World");
System.out.printf("Age = %d\n", 26);
```

Input with Scanner

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.println("Hello " + name + ", age = " + age);
    }
}
```

Common Scanner issue: after `nextInt()/nextDouble()`, the newline can remain in the input buffer. If you then call `nextLine()`, you may need to consume that newline first.

11. Control Flow

Control flow decides which statements execute and how many times they execute.

if / else

```
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

switch

```
int day = 2;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Other");
}
```

Loops



```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

```

int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}

do {
    System.out.println("Runs at least once");
} while (false);

```

Enhanced for loop

```

int[] numbers = {10, 20, 30};

for (int n : numbers) {
    System.out.println(n);
}

```

break exits a loop/switch. continue skips to the next iteration of a loop.

12. Methods

A method groups reusable behavior. It can accept parameters and return a value.

```

public static int add(int a, int b) {
    return a + b;
}

public static void greet(String name) {
    System.out.println("Hello " + name);
}

```

Calling methods:

```

int sum = add(10, 20);
greet("Nisarg");

```

Term	Meaning
Parameter	Variable declared in the method definition.
Argument	Actual value passed when calling the method.
Return type	Type of value returned; void means no value is returned.
Overloading	Multiple methods with the same name but different parameter lists.

13. Arrays and Strings

Arrays

An array stores a fixed number of elements of the same type. Java array indexing starts at 0.

```

int[] marks = {90, 85, 95};

System.out.println(marks[0]);    // 90
System.out.println(marks.length); // 3

```

Strings

String is a class representing text. String objects are immutable: operations that appear to modify a String generally produce a new String value.

```
String first = "Hello";
String second = "Java";

String message = first + " " + second;
System.out.println(message);
System.out.println(message.length());
System.out.println(message.toUpperCase());
```

For String content equality, use equals() rather than == in normal application code.

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a.equals(b)); // true
System.out.println(a == b);      // false
```

14. Object-Oriented Programming Foundations

Four common OOP ideas are encapsulation, inheritance, polymorphism, and abstraction.

Concept	Simple idea	Example
Encapsulation	Keep state and behavior together; control access.	private fields + public methods
Inheritance	A class can derive from another class.	class Dog extends Animal
Polymorphism	One reference can represent different implementations.	Animal a = new Dog();
Abstraction	Expose essential behavior while hiding implementation details.	interface / abstract class

Class and object

```
class Car {
    String model;

    void drive() {
        System.out.println(model + " is driving");
    }
}

public class Demo {
    public static void main(String[] args) {
        Car car = new Car();
        car.model = "Toyota";
        car.drive();
    }
}
```

A class defines structure and behavior; an object is an instance created from that class.

Encapsulation example

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

15. Packages and Access Modifiers

Packages organize related classes and help control visibility.

```
package com.example.app;

import java.util.Scanner;
```

Modifier	Same class	Same package	Subclass	Other package
private	Yes	No	No*	No
(default)	Yes	Yes	No**	No
protected	Yes	Yes	Yes	No***
public	Yes	Yes	Yes	Yes

* Protected/package access has nuances for subclasses in other packages. ** Default/package-private members are not accessible outside the package. *** A protected member can be accessed by a subclass in another package under Java's inheritance rules.

16. Exception Handling

Exceptions represent abnormal conditions that disrupt normal execution. Java provides structured mechanisms for handling them.



```
try {
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero.");
} finally {
    System.out.println("Finished.");
}
```

Checked vs unchecked exceptions

- **Checked exceptions** are subject to compile-time handling/propagation rules.
- **Unchecked exceptions** are RuntimeException subclasses and generally represent programming errors or runtime conditions.

Do not catch Exception blindly just to hide failures. Catch exceptions you can meaningfully handle and preserve useful diagnostic information.

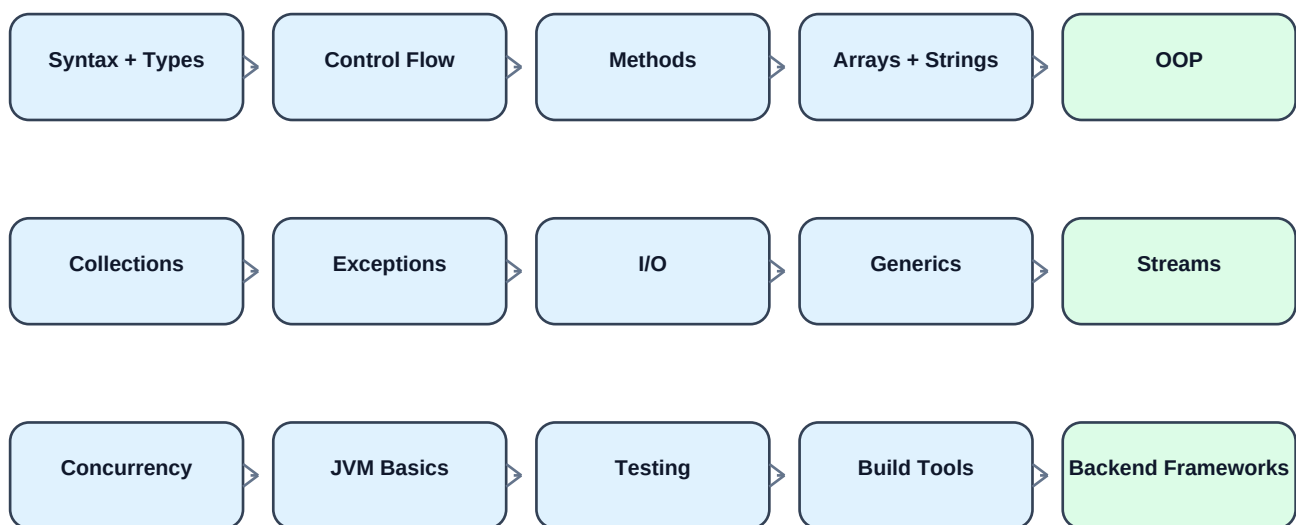
17. Common Beginner Mistakes

Mistake	What to do instead
PATH not configured	Verify java -version and javac -version; configure PATH/JAVA_HOME.
Using == for String contents	Use equals().
Forgetting semicolons	Most Java statements end with ;.

Mistake	What to do instead
Public class/file mismatch	Keep public class name aligned with .java file.
Integer division surprise: $5 / 2 = 2$	Use $5.0 / 2$ when decimal division is intended.
NullPointerException	Check whether a reference is null before dereferencing.
ArrayIndexOutOfBoundsException	Valid indexes are 0 through length - 1.
Confusing compilation/execution	javac compiles; java launches.
Outdated tutorial	Check examples against your JDK version.

18. Practice Roadmap

A practical order for learning Core Java:



Beginner project ideas

- Calculator — variables, operators, methods, input/output.
- Student Grade Manager — arrays/collections, loops, methods, validation.
- Bank Account Simulator — classes, encapsulation, exceptions.
- Library Management System — OOP, collections, search, file I/O.
- Employee Management API — after Core Java, move into Spring Boot and REST.

Learning checkpoints

Stage	Topics	Checkpoint
1	Syntax, variables, primitive types, String	Write small programs without copying syntax.
2	Operators, conditions, loops	Solve 20–30 basic logic problems.
3	Methods, arrays, String APIs	Break a problem into reusable methods.
4	Classes, objects, constructors, encapsulation	Build a small OOP console application.

Stage	Topics	Checkpoint
5	Inheritance, interfaces, polymorphism	Model a multi-class domain.
6	Collections, generics, exceptions, I/O	Build a useful console project.
7	Streams, lambdas, concurrency, JVM basics	Understand modern Java codebases.

Quick command cheat sheet

```
# Check Java
java -version

# Check compiler
javac -version

# Compile
javac HelloWorld.java

# Run
java HelloWorld

# Compile to an output directory
javac -d out HelloWorld.java

# Run from that directory
java -cp out HelloWorld
```

Final mental model



If you understand this flow plus variables, control flow, methods, arrays, and OOP, you have the foundation needed to begin serious Core Java practice.

End of beginner guide